Feature Map Visualization of Pre-trained CNN Models

Name: MD SUMON ALI
Student ID: 2111076159
Department: CSE

February 20, 2026

1 Introduction

Convolutional Neural Networks (CNNs) extract hierarchical features from images. Early layers capture edges and textures, middle layers capture shapes and patterns, and deeper layers capture high-level semantic information.

In this report, we analyze feature maps generated by three pre-trained CNN models:

- VGG16
- ResNet50
- InceptionV3

2 Methodology

A sample RGB image (cat image) was passed through the convolutional base of each pre-trained network. Feature maps were extracted from selected convolutional layers.

All models were loaded with ImageNet pre-trained weights. Only convolutional layers were used for feature visualization.

3 Model 1: VGG16

3.1 Architecture Overview

VGG16 consists of 13 convolutional layers and 3 fully connected layers. It uses small 3×3 convolution filters and uniform architecture.

3.2 Feature Map Observations

- First layers: Detect edges and color gradients.
- Middle layers: Capture textures and object parts.
- Deep layers: Focus on object-specific patterns.

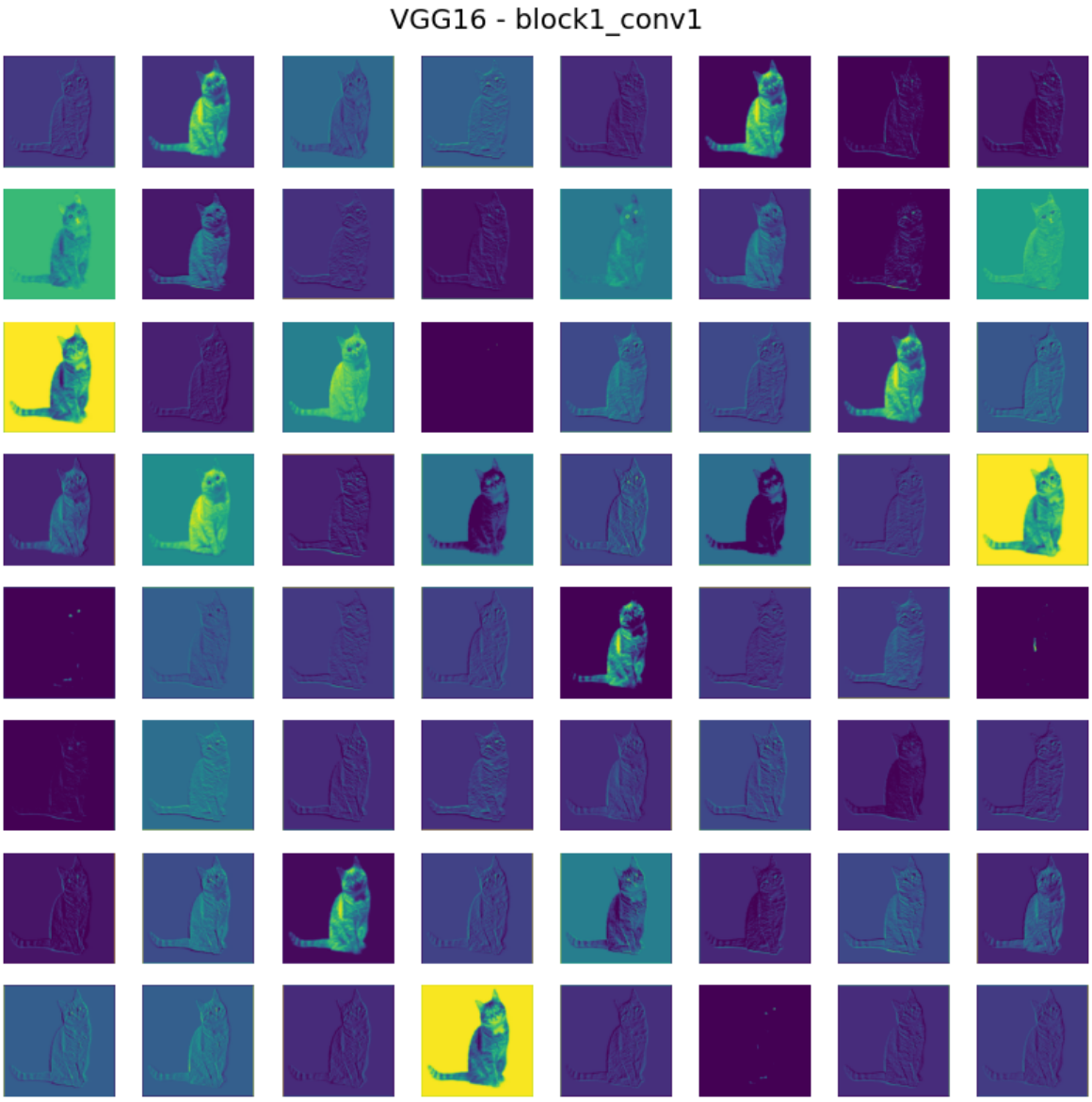

Figure 1: Feature maps from VGG16

4 Model 2: ResNet50

4.1 Architecture Overview

ResNet50 introduces residual connections that help train deeper networks without vanishing gradient problems.

4.2 Feature Map Observations

- Early layers: Edge and corner detection.
- Intermediate layers: Complex texture representation.
- Deeper layers: Semantic abstraction using residual learning.

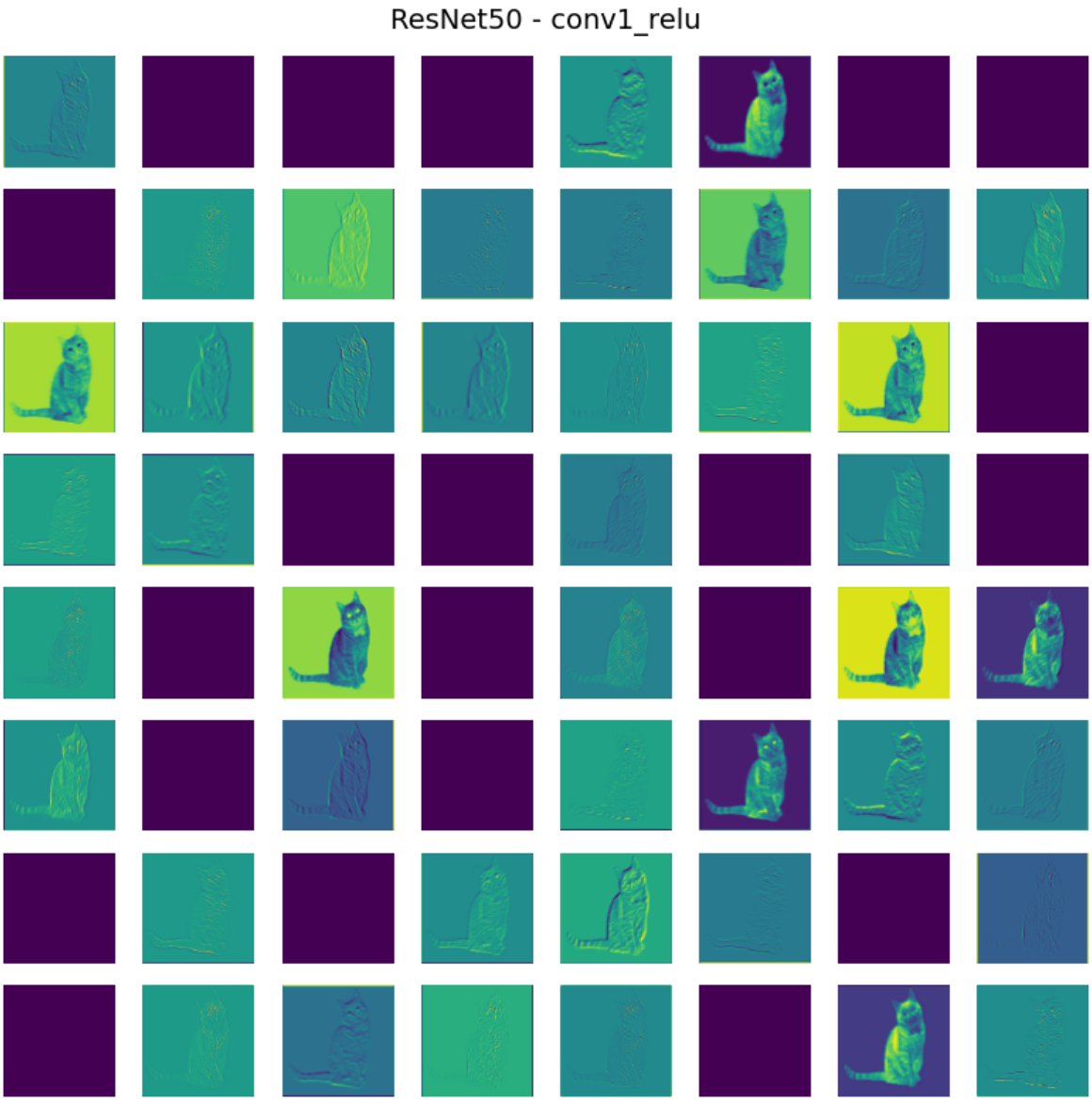

Figure 2: Feature maps from ResNet50

5 Model 3: InceptionV3

5.1 Architecture Overview

InceptionV3 uses parallel convolution filters of different sizes inside Inception modules to capture multi-scale features.

5.2 Feature Map Observations

- Multi-scale feature extraction visible.
- Rich texture and spatial abstraction.
- High-level object representation in deeper layers.

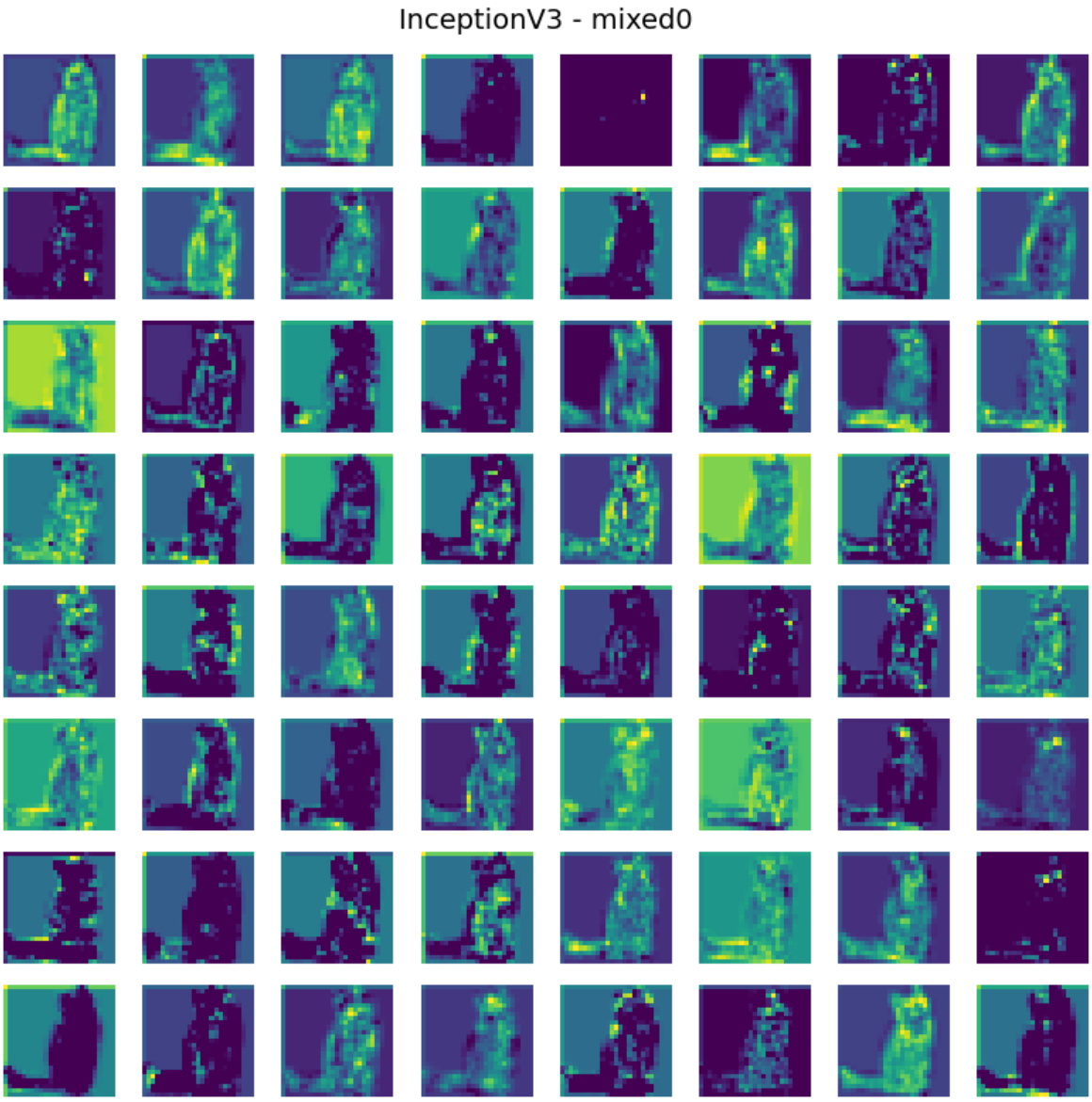

Figure 3: Feature maps from InceptionV3

6 Comparative Analysis

- VGG16: Sequential feature refinement.
- ResNet50: Deeper abstraction due to residual connections.
- InceptionV3: Multi-scale representation in same layer.

7 Conclusion

Feature maps clearly demonstrate hierarchical learning in CNNs. Early layers detect low-level features, while deeper layers capture high-level semantic representations. Residual and Inception architectures provide improved feature abstraction compared to simple sequential CNNs.

8 Appendix: Python Code for Feature Map Extraction

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.applications import VGG16, ResNet50, InceptionV3
from tensorflow.keras.preprocessing import image
from tensorflow.keras.models import Model

img_path = 'cat.jpg'
img = image.load_img(img_path, target_size=(224, 224))
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array = img_array / 255.0


def visualize_feature_maps(base_model, layer_name, model_name, save_name):

    print(f"\nLoading␣{model_name}␣...")

    model = Model(
        inputs=base_model.input,
        outputs=base_model.get_layer(layer_name).output
    )

    feature_maps = model.predict(img_array)

    print(f"{model_name}␣Feature␣map␣shape:", feature_maps.shape)

    num_channels = feature_maps.shape[-1]


    square = int(np.sqrt(num_channels))
    square = min(square, 8)

    plt.figure(figsize=(8, 8))

    for i in range(square * square):
        plt.subplot(square, square, i + 1)
        plt.imshow(feature_maps[0, :, :, i], cmap='viridis')
        plt.axis('off')

    plt.suptitle(f"{model_name}␣-␣{layer_name}", fontsize=14)
    plt.tight_layout()
    plt.savefig(save_name)
    plt.show()

    print(f"{save_name}␣saved␣successfully.")
```

```
#  VGG16

vgg_model = VGG16(weights='imagenet', include_top=False)
visualize_feature_maps(
    base_model=vgg_model,
    layer_name='block1_conv1',
    model_name='VGG16',
    save_name='vgg16_featuremap.png'
)



#  ResNet50

resnet_model = ResNet50(weights='imagenet', include_top=False)
visualize_feature_maps(
    base_model=resnet_model,
    layer_name='conv1_relu',
    model_name='ResNet50',
    save_name='resnet50_featuremap.png'
)



# 3 InceptionV3

inception_model = InceptionV3(weights='imagenet', include_top=False)
visualize_feature_maps(
    base_model=inception_model,
    layer_name='mixed0',
    model_name='InceptionV3',
    save_name='inceptionv3_featuremap.png'
)

print("\nAll feature maps generated successfully!")
```